

Scala Diline Genel Bakış

"Scala" dili anlamını ingilizcede scalable (geliştirilebilir) kelimesinden almaktadır. Scala 3 Book'a göre Scala dilinin en temel özellikleri:

- Yüksek Seviyeli (high-level) bir dildir
- Sentaxı basittir
- Statically typed bir dildir (dynamic hissettirir)
- Fonksiyonel bir programlama dilidir
- Nesne yönelimli bir dildir
- OOP ve FP'nin karışımıdır
- JVM üzerinde ve tarayıcılarda çalışabilir Java ile tam uyumludur
- Term inference oldukça basittir (given, using, extension ...)
- Compile edilen bir dildir
- Genelde sunucu tarafında ve büyük veri setlerinin incelenmesinde kullanılır

Sentax ve Semantik Analiz

2020 tarihinde Scala dilinde sentax olarak sadeliğe gidilmiştir. O yüzden ödevde Scala 3 sentaxı Scala 3 book kullanılarak ele alınacaktır.

Değişkenler

Scalada değişkenler `val` veya `var` ile tanımlanır. `val` immutable olduğu için genelde `val` kullanılır.

```
val msg = "Hello, world"
msg = "Aloha" // hata: reassignment to val (derlenmez)

var count = 0
count = count + 1 // geçerli, var yeniden atanabilir
```

Tip Çıkarımı (Type Inference)

Scala statically typed bir dil olmasına rağmen tip bildirimi çoğu zaman zorunlu değildir. Bu özellik tip çıkarımı (type inference) olarak bilinir ve kodu kısa tutmaya yarar; derleyici çoğunlukla veri tipini çıkarabilir

```
val x = 1 // Int
val s = "a" // String
val nums = List(1, 2, 3) // List[Int]
```

Tip açıkça yazılmak istenirse `:` de kullanılabilir

```
val x: Int = 1
val s: String = "a"
```

dinamik hissettirir derken bu kastedilir. Kısaca tip güvenliği derleyicide statik olarak vardır ve kod yazılırken sürekli tip belirtilmesi zorunlu değildir.

Her Şey Bir İfadedir (Expression)

Scalada kontrol yapıları birer "deyim" (statement) değil bir değer döndüren "ifade"dir (expression). Scala 3 Book, scala dilinin birçok dilden farklı olarak `if / else` yapısının bir değer döndürdüğünü ve bu yüzden onu üçlü operatör (ternary) gibi kullanabileceğini belirtir

```
val x = if a < b then a else b
```

Scala 3'te kontrol yapıları `then`, `do` gibi anahtar kelimelerle ve önemli girinti (significant indentation) ile yazılabilir; bu, C ve Java'daki süslü parantez `{ }` zorunluluğunu büyük ölçüde kaldırır. Örneğin `for` döngüsünde `i <- ints` kodu bir üreteç (generator) olarak adlandırılır ve `do` anahtar kelimesinden sonra gelen kod döngünün gövdesidir.

```
val ints = List(1, 2, 3, 4, 5)
for i <- ints do println(i)
```

Fakat Scalada `for` ifadesi: `do` yerine `yield` anahtar kelimesini kullandığında, sonuç hesaplayıp döndüren `for` ifadeleri oluşturulur. Bu da `for` 'u bir döngüden (yan etki için) bir veri dönüşümüne (yeni değer üreten ifade) çevirir: (Expression durl!)

```
val ints = List(1, 2, 3, 4, 5)
scala> val doubles = for i <- ints yield i * 2
//Scala shell girdisi
val doubles: List[Int] = List(2, 4, 6, 8, 10) //Çıktı
bu şekildedir
```

Scalada `switch` yerine `match` ifadesi kullanılır.

```
val result = i match
  case 1 => "one"
  case 2 => "two"
  case _ => "other"
```

Expression ifadesinin EBNF gösterimi

```
Expr      ::= FunParams ('=>' | '?=>') Expr
           | Expr1
Expr1     ::= 'if' Expr 'then' Expr [ 'else' Expr ]
           | 'while' Expr 'do' Expr
           | 'try' Expr [ 'catch' Expr ] [
'finally' Expr ]
           | 'throw' Expr
           | ForExpr
           | PostfixExpr [ Ascription ]
           | PostfixExpr 'match' MatchClause
FunParams ::= Bindings | id | '_'
Ascription ::= ':' Type
```

OOP ve Scala

Scala, nesne yönelimli programlamayı destekleyen bir dildir. Scala'da **her değer bir sınıfın örneğidir** ve hatta operatörler bile aslında metod çağrısıdır. Scala'da sınıflar, kalıtım ve arayüz benzeri yapılar `class` ve `trait` ile sağlanır. `extends` ile kalıtım sağlanır, `override` ile metotlar ezilir. Çalışma zamanında dinamik bağlama (dynamic dispatch) kullanılır. `private` ve `protected` erişim belirleyicileriyle kapsülleme sağlanır

```

trait Speaker:
  def speak(): String

trait TailWagger:
  def startTail(): Unit = println("tail is wagging")
  def stopTail(): Unit = println("tail is stopped")

trait Runner:
  def startRunning(): Unit = println("I'm running")
  def stopRunning(): Unit = println("Stopped running")

class Dog(name: String) extends Speaker, Runner:
  def speak(): String = "Woof!"

```

Örnek kullanım:

```

val d = new Dog("Fido") with TailWagger
d.startTail() // tail is wagging
d.startRunning() // I'm running

```

Bu örnekte Speaker soyut bir metot (speak) bildirirken, TailWagger ve Runner trait'leri somut metotlar içerir. Dog sınıfı bu üç trait'i birden karıştırır ve yalnızca soyut olan speak metodunu tanımlamak zorundadır. Bu durum genelde Java ile birlikte gelen tekli kalıtım durumunu değiştiren 'mixin' i ele alır.

Aynı zamanda Scala'da Java'daki static anahtar sözcüğü bulunmaz. Bunun yerine sınıfa ait ortak alanlar ve metotlar object içerisinde tanımlanır. Bir sınıfla aynı isimde tanımlanan object , companion object olarak adlandırılır ve Java'daki statik üyelerin karşılığını sağlar:

```

//Scala dilinde
object Utils:
  def square(x: Int): Int = x * x
//-----

```

```

//Java dilinde
class Utils {
  static int square(int x) {
    return x * x;
  }
}
//Ortak kullanım : Utils.square(5)

```

Fonksiyonellik ve Scala

Fonksiyonlar birinci sınıf değerdir (first-class): Scala'da fonksiyonlar birinci sınıf vatandaş olarak ele alınır; tıpkı diğer veri tipleri gibi isimlere bağlanabilir, argüman olarak geçirilebilir ve başka fonksiyonlardan döndürülebilirler. Bu sayede program, küçük fonksiyonların modüler biçimde birleştirildiği bildirimsel (declarative) bir stille yazılabilir.

Yüksek mertebeli fonksiyonlar (HOF): Bir fonksiyonu parametre olarak alan veya döndüren fonksiyonlardır. List sınıfının map metodu, parametre olarak bir fonksiyon alan tipik bir yüksek mertebeli fonksiyon örneğidir.

```

val a = List(1, 2, 3).map(i => i * 2) // List(2, 4, 6)
val b = List(1, 2, 3).map(_ * 2) // List(2, 4, 6)

```

Immutability : Saf fonksiyonel programlamada yalnızca değişmez değerler kullanılır; List, Vector gibi değişmez koleksiyon sınıfları tercih edilir. O yüzden koleksiyonu değiştirmek yerine ona da bir fonksiyon uygulanır ve yeni bir koleksiyon oluşturulur

```

val a = List("john", "jane", "joe", "mary")
val b = a.filter(_.startsWith("j")).map(_.capitalize)
// b = List("John", "Jane", "Joe"); a'ya hiç dokunulmamıştır

```

Saf fonksiyonlar (pure functions): Aynı girdiye her zaman aynı çıktıyı veren ve gözlemlenebilir yan etkisi olmayan fonksiyonlardır; bu da kodun öngörülebilirliğini ve güvenilirliğini artırır.

Alt Programlar

Scala'da alt programlar iki temel biçimde tanımlanır: def ile tanımlanan **metotlar (methods)** ve val ile bir isme bağlanan **fonksiyon değerleri (function values)**. def ifadesi nesnelere bağımlıdır:

```

def topla(a: Int, b: Int): Int = a + b

```

val ise bir Function2[Int, Int, Int] nesnesidir, değişkene atanabilir, parametre olarak geçirilebilir, döndürülebilir:

```

val topla = (a: Int, b: Int) => a + b

```

Bu fonksiyon değerleri tıpkı diğer veriler gibi argüman olarak geçirilebilir veya başka fonksiyonlardan döndürülebilir. Bir metodu fonksiyon değerine dönüştürmek gerektiğinde, eta-genişletme (eta-expansion) adı verilen mekanizma kullanılır:

```

def carp(a: Int, b: Int): Int = a * b
val carpFonksiyonu = carp _ // metot -> fonksiyon değeri

```

Coercion

Scala'nın tip sistemi oldukça katıdır (statically typed), bu yüzden birçok durumda dönüşümü açıkça yapmak gerekir.

```

val x: Int = 42
val y: Long = x // Otomatik dönüşür int -> long
val z: Double = x // Otomatik dönüşür int -> float

```

Tersini yapmak mümkün değildir

```

val x: Double = 3.14
val y: Int = x // derleme hatası
//-----
val y = x.toInt // bu çalışır ama böyle açık açık yazmak gerekir

```

Asıl ilginç olay **kendi coercion mekanizmamızı yazabilmemizdir**. Şu şekilde kullanılır:

```

import scala.language.implicitConversions

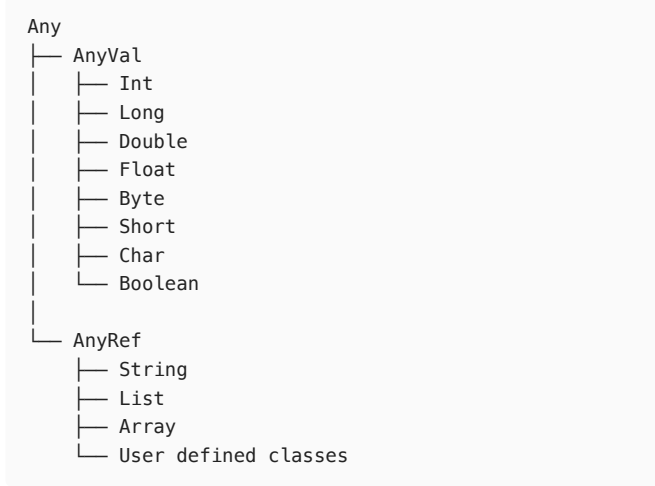
given Conversion[Int, String] with
  def apply(x: Int): String = x.toString

val s: String = 42

```

ADT, Tipler, Generic

Scala'daki tüm tipler en üstte bulunan `Any` sınıfından türemiştir; `AnyVal` değer tiplerini, `AnyRef` ise referans tiplerini temsil eder. Bu yapı sayesinde Scala'da Java'daki primitive-boxed ayrımı kullanıcıya açık şekilde hissettirilmez. Tip hiyerarşisi



Genericlik

Scala'da generic sınıflar veya trait'ler, köşeli parantez `[...]` içinde bir tipi parametre olarak alır.

Örneğin:

```
class Stack[A]:
  private var elements: List[A] = Nil
  def push(x: A): Unit = elements =
    elements.prepend(x)
  def pop(): A = { val t = elements.head; elements =
    elements.tail; t }
```

ADT (Abstract Data Types)

Scalada bu trait ile yapılır

```
trait Stack[A]:
  def push(x: A): Stack[A]
  def pop(): (A, Stack[A])
  def isEmpty: Boolean
```

Binding ve Bellek Yönetimi

Scala'da bir isim bir değere `val` veya `var` ile bağlanır. `val` ile yapılan bağlama değişmezdir (immutable). Üçüncü bir biçim `lazy val` 'dir: değer ilk erişildiği ana kadar hesaplanmaz (tembel değerlendirme), sonra ön belleğe alınır.

```
lazy val greet = {
  println("Hello !")
  val a = 2
  val b = 3
  a + b
}

println(greet) //icerisi calistirilir cikti: Hello !
/n 5
println(greet) //sonuc on bellekte cikti: 5
```

Bağlama **statik kapsam** (lexical scope) kuralına göre çözülür: bir ismin hangi değere karşılık geldiği, kodun yazıldığı bloğun yapısına bakılarak derleme zamanında belirlenir. Buna karşılık, dinamik bağlama (dynamic dispatch) çalışma zamanına aittir: `override` edilmiş bir metodun hangi versiyonunun çağrılacağı, nesnenin çalışma anındaki gerçek tipine göre seçilir.

Bellek Yönetimi: Scala JVM üzerinde çalıştığı için belleği elle yönetmez; bunu JVM otomatik olarak yapar. Yerel değişkenler ve referanslar yığında (stack), nesneler ise öbekte (heap) tutulur. Garbage collection için JVM, Serial GC, Parallel GC, CMS ve G1 gibi çeşitli çöp toplama algoritmaları kullanır. Programcı `new` ile nesne oluşturur ama silmekle uğraşmaz; bir nesneye artık hiçbir referans kalmadığında GC onu otomatik olarak temizler.

Paralellik ve Eşzamanlılık

Scalada paralellik için `Future` yapısı kullanılır. Bir `Future`, şu anda mevcut olabilen veya olmayabilen, ama bir noktada mevcut olacak bir değeri (veya o değer sağlanamazsa bir istisnayı) temsil eder. `Future` içindeki değer her zaman `scala.util.Try` tiplerinden biridir `Success` veya `Failure`. Yani sonucu işlerken olağan `Try` işleme teknikleri kullanılır. herhangi bir şekilde Semaphore veya Fence veya mutex kullanılmaz.

`Future` ifadelerinin birleşip tek bir sonuca almak istiyorsak bunları `for` döngüsünde birleştiririz:

```
// (1) Future döndüren hesaplamaları başlat
val f1 = Future { sleep(800); 1 }
val f2 = Future { sleep(200); 2 }
val f3 = Future { sleep(400); 3 }

// (2) for ifadesinde birleştir
val result =
  for
    r1 <- f1
    r2 <- f2
    r3 <- f3
  yield
    (r1 + r2 + r3)

// (3) sonucu işle
result.onComplete {
  case Success(x) => println(s"result = $x")
  case Failure(e) => e.printStackTrace
}
```

`Future` ifadeleri `onComplete` gibi callbacklerin içlerinin kısmi fonksiyonlarla doldurulmasıyla çağrılır. Semaphore kullanılmaz.

Önemli

Hesaplamalar `for` ifadesinin içinde oluşturulursa işlemler paralel değil, ardışık olarak çalışır. Bu nedenle paralellik isteniyorsa `Future` nesneleri `for` döngüsünün **dışında** başlatılmalıdır.

video linki :

https://drive.google.com/file/d/1TeT8UITMD3wrUcXv5aP1KEjBWQtg6SnB/view?usp=share_link

Ahmet Sacid Aktan
24360859020